

UNIP

CAUÃ ROCHA DE MORAES

PROGRAMAÇÃO ORIENTADA A OBJETOS

BRASÍLIA-DF 2025

Código Fonte:

```
import java.util.Scanner;

public class olaUsuario {
    public static void main(String[] args) {
        // Criando o objeto Scanner para ler a entrada do
        teclado
        Scanner leitor = new Scanner(System.in);

        // Solicitação de dados básica
        System.out.print("Digite seu nome: ");
        String nome = leitor.nextLine();

        // Exibindo a mensagem personalizada inicial
        System.out.println("Olá " + nome + ", bem-vindo
        ao mundo da programação em Java!");

        System.out.println("\n--- Ampliação da Atividade
        ---");

        // Solicitando informações adicionais
        System.out.print("Digite sua idade: ");
        int idade = leitor.nextInt();
import java.util.Scanner;

public class olaUsuario {
    public static void main(String[] args) {
        // Criando o objeto Scanner para ler a entrada do
        teclado
        Scanner leitor = new Scanner(System.in);

        // Solicitação de dados básica
        System.out.print("Digite seu nome: ");
        String nome = leitor.nextLine();

        // Exibindo a mensagem personalizada inicial
        System.out.println("Olá " + nome + ", bem-vindo
        ao mundo da programação em Java!");
```

```

        System.out.println("\n--- Ampliação da Atividade
---");

        // Solicitando informações adicionais
        System.out.print("Digite sua idade: ");
        int idade = leitor.nextInt();

        // Limpar o buffer do teclado
        leitor.nextLine();

        System.out.print("Digite seu curso: ");
        String curso = leitor.nextLine();

        // Exemplo de saída final formatada
        System.out.println("\nOlá " + nome + "!");
        System.out.println("Você tem " + idade + "
anos.");
        System.out.println("Bem-vindo ao curso de " +
curso + ".");

        // Fechando o leitor
        leitor.close();
    }
}

// Limpar o buffer do teclado
leitor.nextLine();

System.out.print("Digite seu curso: ");
String curso = leitor.nextLine();

// Exemplo de saída final formatada
System.out.println("\nOlá " + nome + "!");
System.out.println("Você tem " + idade + "
anos.");
System.out.println("Bem-vindo ao curso de " +
curso + ".");

// Fechando o leitor
leitor.close();
}

```

```
}
```

Explicação sobre JAVA e o Programa Desenvolvido:

O que é o JAVA?

Java é uma linguagem de programação orientada a objetos criada para desenvolver programas, sistemas e aplicações. Ela é muito utilizada no desenvolvimento de aplicativos, sistemas empresariais, jogos e aplicações web.

Uma das principais características do Java é sua portabilidade, pois os programas podem ser executados em diferentes sistemas operacionais através da Máquina Virtual Java (JVM).

O que é uma IDE

IDE significa Integrated Development Environment (Ambiente de Desenvolvimento Integrado). É um programa utilizado para criar, editar, organizar e executar códigos de programação.

Exemplos de IDEs utilizadas em Java

- IntelliJ IDEA
- Eclipse IDE
- NetBeans

A IDE facilita o desenvolvimento, pois possui:

- Editor de códigos;
- Correção de erros;
- Compilação automática;
- Execução do Programa;
- Organização de arquivo.

Explicação de como o programa foi criado

O programa foi desenvolvido com o objetivo de praticar:

- entrada de dados;
- armazenamento em variáveis;
- exibição de informações na tela;
- interação com o usuário.

Primeiramente, foi importada a biblioteca Scanner para permitir a leitura pelo teclado.

Depois:

Foi criada a classe principal;

Foi criado o método main;

Foi instanciado o objeto Scanner;

O programa passou a solicitar informações do usuário:

nome;

idade;

curso;

As informações digitadas foram armazenadas em variáveis;

Por fim, os dados foram exibidos em mensagens personalizadas.

Passos realizados na IDE

1. Criação do projeto

Foi criado um projeto Java na IDE escolhida.

2. Criação da classe

Foi criada uma classe chamada:

```
olaUsuario
```

3. Digitação do código

O código foi escrito dentro do método main.

4. Importação da biblioteca Scanner

Foi adicionada a linha:

```
import java.util.Scanner;
```

para permitir a entrada de dados pelo teclado.

5. Compilação do programa

A IDE compilou o código para verificar possíveis erros de sintaxe.

6. Execução do programa

Após compilar corretamente, o programa foi executado.

O usuário digitou:

nome;

idade;

curso.

E o sistema exibiu as mensagens personalizadas na tela.

Conclusão

O programa desenvolvido demonstra conceitos básicos da programação em Java, como:

utilização de variáveis;

entrada e saída de dados;

criação de classes;

uso do método principal;

interação com o usuário.

Além disso, a atividade permitiu compreender o funcionamento de uma IDE e o processo básico de criação e execução de programas em Java.

UNIP

CAUÃ ROCHA DE MORAES

PROGRAMAÇÃO ORIENTADA A OBJETOS

BRASÍLIA-DF 2025

Código Fonte:

```
import java.util.Scanner;

public class SistemaEscolar {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        int opcao = 0;

        while (opcao != 3) {
            System.out.println("\n--- MENU INTERATIVO ---");

            System.out.println("1. Iniciar Cadastro de Alunos (Desafio Extra)");
            System.out.println("2. Verificar Idade e Nota Individual");
            System.out.println("3. Sair");
            System.out.print("Escolha uma opção: ");
            opcao = leitor.nextInt();
            leitor.nextLine(); // Limpar buffer

            switch (opcao) {
                case 1:
                    processarCincoAlunos(leitor);
                    break;
                case 2:
                    processarIndividual(leitor);
                    break;
                case 3:
                    System.out.println("Encerrando o sistema...");
                    break;
                default:
                    System.out.println("Opção inválida!");
            }
        }
    }
}
```



```

        }
    }
    leitor.close();
}

// Nível 5: Desafio Extra (5 Alunos)
public static void processarCincoAlunos(Scanner
leitor) {
    int aprovados = 0;
    for (int i = 1; i <= 5; i++) {
        System.out.println("\n--- Aluno " + i + " ---
");
        System.out.print("Nota: ");
        double nota = leitor.nextDouble();
        if (nota >= 7) aprovados++;
    }
    System.out.println("\nTotal de alunos aprovados:
" + aprovados);
}

// Níveis 1, 2, 3 e 4 integrados
public static void processarIndividual(Scanner
leitor) {
    // Validação de Nome
    System.out.print("Nome: ");
    String nome = leitor.nextLine();

    // Validação de Idade (Nível 1)
    int idade;
    do {
        System.out.print("Idade (maior que 0): ");
        idade = leitor.nextInt();
    } while (idade <= 0);

    // Validação de Nota e Média (Nível 4)
    double somaNotas = 0;
    System.out.print("Quantas notas deseja inserir
para calcular a média? ");
    int qtdNotas = leitor.nextInt();

    for (int i = 1; i <= qtdNotas; i++) {

```

```

        double nota;
        do {
            System.out.print("Nota " + i + " (0 a
10): ");
            nota = leitor.nextDouble();
        } while (nota < 0 || nota > 10);
        somaNotas += nota;
    }import java.util.Scanner;

public class SistemaEscolar {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        int opcao = 0;

        while (opcao != 3) {
            System.out.println("\n--- MENU INTERATIVO ---
");
            System.out.println("1. Iniciar Cadastro de
Alunos (Desafio Extra)");
            System.out.println("2. Verificar Idade e Nota
Individual");
            System.out.println("3. Sair");
            System.out.print("Escolha uma opção: ");
            opcao = leitor.nextInt();
            leitor.nextLine(); // Limpar buffer

            switch (opcao) {
                case 1:
                    processarCincoAlunos(leitor);
                    break;
                case 2:
                    processarIndividual(leitor);
                    break;
                case 3:
                    System.out.println("Encerrando o
sistema...");
                    break;
                default:
                    System.out.println("Opção
inválida!");
            }
        }
    }
}

```

```

        leitor.close();
    }

    // Nível 5: Desafio Extra (5 Alunos)
    public static void processarCincoAlunos(Scanner
leitor) {
        int aprovados = 0;
        for (int i = 1; i <= 5; i++) {
            System.out.println("\n--- Aluno " + i + " ---
");
            System.out.print("Nota: ");
            double nota = leitor.nextDouble();
            if (nota >= 7) aprovados++;
        }
        System.out.println("\nTotal de alunos aprovados:
" + aprovados);
    }

    // Níveis 1, 2, 3 e 4 integrados
    public static void processarIndividual(Scanner
leitor) {
        // Validação de Nome
        System.out.print("Nome: ");
        String nome = leitor.nextLine();

        // Validação de Idade (Nível 1)
        int idade;
        do {
            System.out.print("Idade (maior que 0): ");
            idade = leitor.nextInt();
        } while (idade <= 0);

        // Validação de Nota e Média (Nível 4)
        double somaNotas = 0;
        System.out.print("Quantas notas deseja inserir
para calcular a média? ");
        int qtdNotas = leitor.nextInt();

        for (int i = 1; i <= qtdNotas; i++) {
            double nota;
            do {

```

```

        System.out.print("Nota " + i + " (0 a
10): ");

        nota = leitor.nextDouble();
    } while (nota < 0 || nota > 10);
    somaNotas += nota;
}
double media = somaNotas / qtdNotas;

// Saída de Dados
System.out.println("\n--- RESULTADO ---");
System.out.println("Nome: " + nome);
System.out.println("Maior de idade: " + (idade >=
18 ? "Sim" : "Não"));

// Classificação da Nota
if (media >= 7) {
    System.out.println("Situação: Aprovado
(Média: " + media + ")");
} else if (media >= 5) {
    System.out.println("Situação: Recuperação
(Média: " + media + ")");
} else {
    System.out.println("Situação: Reprovado
(Média: " + media + ")");
}

// Contagem (Laço de Repetição)
System.out.print("Contagem até sua idade: ");
for (int i = 1; i <= idade; i++) {
    System.out.print(i + " ");
}
System.out.println();
}
}

double media = somaNotas / qtdNotas;

// Saída de Dados
System.out.println("\n--- RESULTADO ---");
System.out.println("Nome: " + nome);
System.out.println("Maior de idade: " + (idade >=
18 ? "Sim" : "Não"));

```

```

        // Classificação da Nota
        if (media >= 7) {
            System.out.println("Situação: Aprovado
(Média: " + media + ")");
        } else if (media >= 5) {
            System.out.println("Situação: Recuperação
(Média: " + media + ")");
        } else {
            System.out.println("Situação: Reprovado
(Média: " + media + ")");
        }

        // Contagem (Laço de Repetição)
        System.out.print("Contagem até sua idade: ");
        for (int i = 1; i <= idade; i++) {
            System.out.print(i + " ");
        }
        System.out.println();
    }
}

```

Introdução

Conceito de Controle de Fluxo:

O controle de fluxo é o mecanismo que determina a ordem em que as instruções de um programa são executadas. Sem ele, um computador apenas processaria linhas de código linearmente, do topo ao fundo. Com o controle de fluxo, introduzimos a tomada de decisão e a automação de tarefas repetitivas, permitindo que o software reaja a diferentes entradas e condições.

Estruturas Condicionais e de Repetição

Estruturas Condicionais (if, else, switch): Funcionam como bifurcações no caminho. Elas permitem que o programa execute um bloco de código apenas se uma condição específica (verdadeira ou falsa) for atendida.

Estruturas de Repetição (for, while, do-while): Também conhecidas como loops ou laços, essas estruturas permitem que um bloco de código seja executado múltiplas vezes enquanto uma condição permanecer válida, evitando a redundância de escrita.

2. Desenvolvimento

Explicação do Programa

Dica: Aqui você deve descrever o que o seu programa faz na prática. Abaixo, deixo um modelo que você pode preencher:

O programa desenvolvido tem como objetivo [inserir objetivo, ex: calcular a média de alunos / gerenciar um estoque]. A lógica principal baseia-se em receber dados do usuário via [inserir entrada, ex: teclado/terminal], processar esses dados através de verificações lógicas e retornar um resultado condizente com a situação apresentada.

O fluxo de execução foi desenhado para ser intuitivo: o sistema inicia coletando os dados necessários, valida se as informações são consistentes e, caso positivo, entra na etapa de processamento pesado utilizando laços de repetição para tratar volumes de dados ou aguardar uma ação final do usuário.

Estruturas Utilizadas

Nesta seção, detalhamos como a teoria foi aplicada na prática:

Condicionais (if/else): Foram aplicadas para realizar a validação de dados. Por exemplo, se o valor inserido for menor que zero, o programa emite um alerta; caso contrário, segue para o cálculo.

Exemplo: `if (valor > limite) { ... }`

Repetição (for ou while): Utilizou-se a estrutura [escolha uma] para iterar sobre uma lista de elementos ou repetir o menu principal até que o usuário decida encerrar o programa (condição de parada).

Exemplo: O laço while foi fundamental para manter o programa rodando continuamente, criando uma interface dinâmica.

UNIP

CAUÃ ROCHA DE MORAES

PROGRAMAÇÃO ORIENTADA A OBJETOS

BRASÍLIA-DF 2025

Código Fonte:

```
import java.util.Scanner;

class Funcionario {
    private String nome;
    private double salario;

    // Nível 2: Sobrecarga de Construtores
    public Funcionario() {}

    public Funcionario(String nome, double salario) {
        this.nome = nome;
        setSalario(salario); // Usa o setter para validar
    }

    // Getters e Setters com Validação (Nível 1)
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public double getSalario() { return salario; }
    public void setSalario(double salario) {
        if (salario >= 0) {
            this.salario = salario;
        } else {
            System.out.println("Erro: Salário não pode ser negativo!");
        }
    }

    public void exibirDados() {
        System.out.println("Nome: " + nome + " | Salário Base: R$" + salario);
    }
}

// --- CLASSE DERIVADA (Herança e Sobrescrita) ---
class Gerente extends Funcionario {
    private double bonus;
```



```

    public Gerente(String nome, double salario, double bonus)
    {
        super(nome, salario); // Nível 2: Uso do super
        this.bonus = bonus;
    }

    // Nível 5: Cálculo de salário com bônus
    @Override
    public double getSalario() {
        return super.getSalario() + bonus;
    }

    @Override
    public void exibirDados() {
        System.out.println("[GERENTE] Nome: " + getNome() +
            " | Salário Total (c/ bônus): R$" +
getSalario());
    }
}

// --- CLASSE DERIVADA (Nível 3: Hierarquia) ---
class Estagiario extends Funcionario {
    public Estagiario(String nome, double salario) {
        super(nome, salario);
    }

    @Override
    public void exibirDados() {
        System.out.println("[ESTAGIÁRIO] Nome: " + getNome() +
" | Bolsa: R$" + getSalario());
    }
}

// --- CLASSE PRINCIPAL (Nível 5: Menu Interativo) ---
public class SistemaRH {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);

        // Nível 4: Polimorfismo (Referência da classe base
para objetos derivados)
        Funcionario func1 = new Gerente("Aline Rocha", 5000,
1500);
        Funcionario func2 = new Estagiario("Matheus Silva",
1200);

```

```

        System.out.println("--- Demonstração de Polimorfismo -
--");

        func1.exibirDados();
        func2.exibirDados();

        // Menu Interativo
        int opcao = 0;
        while (opcao != 2) {
            System.out.println("\n1. Cadastrar Novo
Gerente\n2. Sair");
            System.out.print("Escolha: ");
            opcao = leitor.nextInt();
            leitor.nextLine();

            if (opcao == 1) {
                System.out.print("Nome: ");
                String n = leitor.nextLine();
                System.out.print("Salário: ");
                double s = leitor.nextDouble();
                System.out.print("Bônus: ");
                double b = leitor.nextDouble();

                Gerente g = new Gerente(n, s, b);
                g.exibirDados();
            }
        }
        leitor.close();
    }
}

```

1. Introdução

Conceitos de POO

A Programação Orientada a Objetos (POO) é um paradigma de programação baseado no conceito de "objetos", que podem conter dados na forma de campos (atributos) e códigos na forma de procedimentos (métodos). Diferente da programação estruturada, que foca em funções e lógica sequencial, a POO organiza o software em torno de entidades do mundo real, facilitando a reutilização de código e a manutenção de sistemas complexos.

Encapsulamento e Herança

Estes são dois dos quatro pilares fundamentais da POO:

- **Encapsulamento:** É a técnica de "esconder" os detalhes internos de funcionamento de um objeto, expondo apenas o que é necessário através de uma interface pública. Isso é feito utilizando modificadores de acesso (como `private`, `protected` e `public`), garantindo que os dados internos não sejam corrompidos por acesso externo indevido.
- **Herança:** Permite que uma classe (filha/subclasse) herde atributos e métodos de outra classe (pai/superclasse). Isso promove o reuso de código e estabelece uma relação de "é um", onde a classe filha é uma especialização da classe pai.

2. Desenvolvimento

Explicação das Classes Criadas

Nesta seção, o programa desenvolvido é detalhado através de suas unidades fundamentais. As classes implementadas servem como moldes para os objetos que compõem o sistema.

- **Classe [Nome da Superclasse]:** Atua como a base do sistema, contendo os atributos genéricos como [**ex: nome, id, data**] e métodos que são comuns a todas as especializações.
- **Classe [Nome da Subclasse]:** Implementa comportamentos específicos que não existem na classe base, demonstrando como a estrutura foi moldada para atender aos requisitos do problema.
- **Métodos e Comportamentos:** Foram implementados métodos para [**ex: calcular valores, formatar saída de dados**], garantindo que a lógica de negócio esteja contida dentro do objeto responsável por ela.

Hierarquia Implementada

A hierarquia foi construída para refletir a organização lógica dos dados. No topo da estrutura, temos a classe mais abstrata, que define o contrato básico do objeto.

Abaixo dela, as subclasses derivam seus comportamentos, o que permite:

1. **Especialização:** Cada subnível adiciona complexidade e detalhes específicos.
2. **Polimorfismo:** (Caso tenha usado) A capacidade de tratar objetos de diferentes subclasses como se fossem da classe pai, permitindo que um único método se comporte de formas diferentes dependendo do objeto que o chama.

Exemplo Prático: No projeto, a hierarquia impede a duplicação de código, pois características comuns a todos os elementos foram centralizadas na classe pai, enquanto variações foram tratadas exclusivamente nas classes filhas.

UNIP

CAUÃ ROCHA DE MORAES

PROGRAMAÇÃO ORIENTADA A OBJETOS

BRASÍLIA-DF 2025

Código Fonte:

```
import java.util.ArrayList;
import java.util.Scanner;

// Nível 5: Classe de objeto para armazenar na lista
class Aluno {
    private String nome;

    public Aluno(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}

public class GestaoCompleta {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);

        // Listas para diferentes propósitos
        ArrayList<Aluno> listaAlunos = new ArrayList<>();
        // Lista de Objetos
        ArrayList<Double> listaNotas = new ArrayList<>();
        // Nível 2: Lista de Números

        int opcao = 0;

        while (opcao != 5) {
            System.out.println("\n===== SISTEMA
ACADÊMICO =====");
            System.out.println("1. Adicionar Aluno
(Objeto)");
            System.out.println("2. Listar Alunos e
Remover");
            System.out.println("3. Lançar Notas e
Calcular Média (Nível 2)");
            System.out.println("4. Exibir Relatório de
Notas");
```

```

        System.out.println("5. Sair");
        System.out.print("Escolha uma opção: ");

        opcao = leitor.nextInt();
        leitor.nextLine(); // Limpeza de buffer

        switch (opcao) {
            case 1:
                // Nível 1: Entrada dinâmica
                System.out.print("Digite o nome do
aluno: ");

                String nomeParaAdd =
leitor.nextLine();
                listaAlunos.add(new
Aluno(nomeParaAdd));
                System.out.println("Aluno adicionado
com sucesso!");
                break;

            case 2:
                if (listaAlunos.isEmpty()) {
                    System.out.println("A lista está
vazia.");
                } else {
                    System.out.println("\n--- Lista
de Alunos ---");

                    for (int i = 0; i <
listaAlunos.size(); i++) {
                        System.out.println(i + " - "
+ listaAlunos.get(i).getNome());
                    }

                    System.out.print("\nDeseja
remover algum? Digite o índice ou -1 para cancelar: ");
                    int indice = leitor.nextInt();
                    if (indice >= 0 && indice <
listaAlunos.size()) {
                        Aluno removido =
listaAlunos.remove(indice);
                        System.out.println("Aluno " +
removido.getNome() + " removido.");
                    }
                }
        }

```

```

        break;

        case 3:
            // Nível 2: Lista de números, soma e
média
            System.out.println("\n--- Lançamento
de Notas ---");
            System.out.print("Quantas notas
deseja inserir? ");
            int qtd = leitor.nextInt();

            for (int i = 1; i <= qtd; i++) {
                System.out.print("Digite a nota "
+ i + ": ");
                double nota =
leitor.nextDouble();
                listaNotas.add(nota);
            }

            // Cálculo da média
            double soma = 0;
            for (double n : listaNotas) {
                soma += n;
            }
            double media = listaNotas.isEmpty() ?
0 : soma / listaNotas.size();
            System.out.printf("Média das notas
inseridas: %.2f\n", media);
            break;

        case 4:
            System.out.println("\n--- Relatório
de Notas (Nível 2) ---");
            if (listaNotas.isEmpty()) {
                System.out.println("Nenhuma nota
lançada.");
            } else {
                System.out.println("Notas: " +
listaNotas);
                System.out.println("Total de
registros: " + listaNotas.size());
            }
            break;

```



```

        case 5:
            System.out.println("Encerrando o
sistema. Até logo!");
            break;

        default:
            System.out.println("Opção
inválida!");
    }
}
leitor.close();
}
}

```

1. Introdução

Conceito de Listas Dinâmicas

Diferente dos arrays (vetores) convencionais, que possuem um tamanho fixo definido no momento da criação, as **listas dinâmicas** têm a capacidade de redimensionar-se automaticamente. Elas permitem a inserção e remoção de elementos em tempo de execução, gerenciando a memória de forma eficiente para acomodar apenas a quantidade necessária de dados. São estruturas fundamentais quando não se conhece previamente o volume total de informações que o sistema irá processar.

Diferença entre ArrayList e Vector

Embora ambas as classes implementem a interface List e utilizem um array interno para armazenar dados, elas possuem diferenças cruciais de desempenho e comportamento:

Característica	ArrayList	Vector
Sincronização	Não é sincronizada (Thread-Safe: Não).	É sincronizada (Thread-Safe: Sim).
Desempenho	Mais rápida, pois não possui sobrecarga de trava de threads.	Mais lenta devido ao controle de sincronização.

Crescimento	Aumenta em cerca de 50% do tamanho atual quando lotada.	Dobra de tamanho (100%) quando lotada.
Uso Recomendado	Aplicações modernas e sistemas de thread única.	Sistemas legados ou onde a segurança de threads é crítica.

2. Desenvolvimento

Explicação do Programa

O programa desenvolvido simula um sistema de **[ex: Cadastro de Alunos]**, utilizando uma lista dinâmica para gerenciar os registros. O fluxo consiste em permitir que o usuário adicione nomes à lista, verifique a existência de determinados itens e remova registros obsoletos. A escolha por uma lista dinâmica justifica-se pela natureza variável do número de alunos, evitando o desperdício de memória ou o estouro de limite de capacidade.

Métodos Utilizados

Para a manipulação da lista, foram utilizados os seguintes métodos nativos da classe:

- **add(E element)**: Insere um novo objeto ao final da lista.
- **remove(int index) / remove(Object o)**: Remove um item específico da lista, reorganizando os índices subsequentes.
- **size()**: Retorna a quantidade atual de elementos presentes na estrutura.
- **get(int index)**: Recupera o valor armazenado em uma posição específica.
- **contains(Object o)**: Verifica se um determinado elemento já existe na lista (retorna booleano).

UNIP

CAUÃ ROCHA DE MORAES

PROGRAMAÇÃO ORIENTADA A OBJETOS

BRASÍLIA-DF 2025

1. Introdução

Conceito de Polimorfismo

O termo **Polimorfismo** tem origem grega (*poly* = muitas, *morphe* = formas). Na POO, ele representa a capacidade de um objeto ser referenciado de várias formas. Em termos práticos, é o princípio que permite que uma classe pai aponte para um objeto de uma classe filha, e que diferentes classes derivadas respondam à mesma mensagem (chamada de método) de maneiras distintas, de acordo com seu comportamento específico.

Tipos de Polimorfismo

Existem dois tipos principais utilizados no desenvolvimento de software:

1. **Polimorfismo de Sobrecarga (Estático / Compilação):** Ocorre quando temos métodos com o mesmo nome na mesma classe, mas com assinaturas (parâmetros) diferentes. O compilador decide qual método chamar com base nos argumentos passados.
2. **Polimorfismo de Sobreposição (Dinâmico / Execução):** Ocorre quando uma subclasse redefine um método da superclasse (usando a anotação `@Override`). A decisão de qual método será executado acontece apenas durante a execução do programa (*runtime*).

2. Desenvolvimento

Explicação do Sistema Desenvolvido

O sistema foi projetado para demonstrar a flexibilidade do polimorfismo através de um [ex: **Sistema de Pagamentos / Simulador de Animais**]. O objetivo central foi criar uma estrutura onde o programa principal interage com uma classe genérica, sem precisar conhecer os detalhes específicos de cada implementação filha.

Isso permite que o sistema seja facilmente expansível: se uma nova funcionalidade for adicionada futuramente, o código principal não precisará de alterações drásticas, pois ele já está preparado para lidar com qualquer objeto que siga a "forma" da classe base.

Classes Criadas

A estrutura do projeto conta com as seguintes definições:

- **Classe Abstrata / Interface [Nome da Classe Pai]:** Define o método base, como por exemplo `calcular()` ou `emitirSom()`. Ela serve como o contrato comum para todas as outras classes.
- **Classe [Nome da Filha 1]:** Implementa o método base de forma específica para a sua realidade.
 - *Exemplo:* Se a classe pai é `Pagamento`, esta classe filha poderia ser `CartaoCredito`, aplicando as taxas correspondentes ao cartão.

- **Classe [Nome da Filha 2]:** Implementa o mesmo método, porém com uma lógica totalmente diferente.
 - *Exemplo:* Seguindo a analogia, esta seria a classe Pix, que processa o pagamento de forma instantânea e sem taxas de operadora.

Destaque Técnico: No código, o polimorfismo foi observado quando criamos uma lista de objetos do tipo pai e, ao percorrê-la, cada objeto executou sua própria versão do método, evidenciando o comportamento dinâmico do sistema

Código Fonte:

```
import java.util.ArrayList;
import java.util.Scanner;

// =====
// CLASSES DE MODELO
// =====

abstract class FuncionarioApp {
    private String nome;
    protected double salarioBase;

    public FuncionarioApp(String nome, double
salarioBase) {
        this.nome = nome;
        this.salarioBase = salarioBase;
    }

    public String getNome() { return nome; }
    public abstract double calcularSalario();
}

class GerenteApp extends FuncionarioApp {
    public GerenteApp(String nome, double salarioBase) {
        super(nome, salarioBase);
    }
    @Override
    public double calcularSalario() {
        return salarioBase + 2000.0;
    }
}
```

```

}

class DesenvolvedorApp extends FuncionarioApp {
    public DesenvolvedorApp(String nome, double
salarioBase) {
        super(nome, salarioBase);
    }
    @Override
    public double calcularSalario() {
        return salarioBase;
    }
}

class EstagiarioApp extends FuncionarioApp {
    public EstagiarioApp(String nome, double salarioBase)
{
        super(nome, salarioBase);
    }
    @Override
    public double calcularSalario() {
        return salarioBase * 0.8;
    }
}

// =====
// CLASSE PRINCIPAL
// =====

public class SistemaRHCompleto {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        ArrayList<FuncionarioApp> funcionarios = new
ArrayList<>();
        int opcao = 0;

        while (opcao != 3) {
            System.out.println("\n--- GESTAO RH ---");
            System.out.println("1. Cadastrar");
            System.out.println("2. Relatorio (Folha de
Pagamento)");
            System.out.println("3. Sair");

```

```

        System.out.print("Escolha: ");

        if (!leitor.hasNextInt()) {
            leitor.next(); // Limpa erro
            continue;
        }
        opcao = leitor.nextInt();
        leitor.nextLine();

        if (opcao == 1) {
            System.out.println("1-Gerente, 2-Dev, 3-
Estagiario");
            int tipo = leitor.nextInt();
            leitor.nextLine();

            System.out.print("Nome: ");
            String nome = leitor.nextLine();
            System.out.print("Salario Base: ");
            double base = leitor.nextDouble();

            // Criando objetos com os novos nomes de
classe
            if (tipo == 1) funcionarios.add(new
GerenteApp(nome, base));
            else if (tipo == 2) funcionarios.add(new
DesenvolvedorApp(nome, base));
            else if (tipo == 3) funcionarios.add(new
EstagiarioApp(nome, base));

            System.out.println("Cadastrado!");

        } else if (opcao == 2) {
            System.out.println("\n--- FOLHA DE
PAGAMENTO ---");
            if (funcionarios.isEmpty()) {
                System.out.println("Lista vazia.");
            } else {
                for (FuncionarioApp f : funcionarios)
                {
                    // POLIMORFISMO: chama o
calcularSalario correto

```

```

                                System.out.printf("Nome: %-10s |
Total: R$ %.2f\n",
                                f.getNome(),
f.calcularSalario());
                                }
                                }
                                }
                                }
                                leitor.close();
                                }

```


UNIP

CAUÃ ROCHA DE MORAES

PROGRAMAÇÃO ORIENTADA A OBJETOS

BRASÍLIA-DF 2025

1. Introdução

Conceito de Modificadores

Modificadores de acesso são palavras-chave utilizadas na programação orientada a objetos para definir a visibilidade e o nível de acessibilidade de classes, atributos e métodos. Eles funcionam como "portões" que controlam quais partes do código podem interagir com determinados membros de um objeto, protegendo a estrutura interna contra usos indevidos ou acidentais.

Importância na POO

O uso de modificadores é a base para o **Encapsulamento**. Sem eles, qualquer parte do sistema poderia alterar valores críticos de um objeto (como o saldo de uma conta bancária), o que tornaria o software vulnerável e difícil de manter. Sua importância reside em:

- **Segurança:** Protege dados sensíveis.
- **Abstração:** Esconde a complexidade interna, expondo apenas o necessário.
- **Manutenção:** Permite alterar a lógica interna de uma classe sem quebrar o restante do sistema.

2. Desenvolvimento

Explicação do Sistema Criado

O sistema desenvolvido consiste em um [ex: **Gerenciador de Contas de Usuários**], onde a proteção da informação é vital. O fluxo foi estruturado para que os dados brutos nunca sejam acessados diretamente por outras classes. Em vez disso, o sistema utiliza métodos intermediários para validar e processar qualquer alteração, garantindo que as regras de negócio sejam sempre respeitadas.

Uso de cada Modificador

No projeto, os modificadores foram aplicados da seguinte forma:

Modificador	Aplicação no Projeto	Justificativa
public	Classes e métodos de interface (ex: <code>exibirDados()</code>).	Permite que qualquer outra parte do programa utilize as funcionalidades principais.

private	Atributos sensíveis (ex: senha, saldo, id).	Restringe o acesso apenas à própria classe, impedindo modificações externas diretas.
protected	Atributos de classes pai (ex: nomeUsuario).	Permite que apenas a classe original e suas subclasses (herança) acessem o dado, mantendo-o oculto para o restante.
default (pacote)	Classes auxiliares dentro do mesmo diretório.	Utilizado quando a visibilidade é necessária apenas entre classes que trabalham juntas no mesmo módulo.

Nota Técnica: Para acessar os atributos definidos como `private`, foram implementados métodos **Getters e Setters**. Isso permitiu adicionar validações (como: "não permitir saldo negativo") antes de efetivamente alterar o valor do atributo.

Código Fonte:

```
import java.util.ArrayList;
import java.util.Scanner;

// =====
// CAMADA DE MODELO (Abstração e Regras)
// =====

abstract class FuncionarioBase {
    private String nome;

    // Nível 1: Constantes (final e static)
    public static final double SALARIO_MINIMO = 1412.00;
    public static final double TAXA_BONUS_GERENTE = 0.20;
    // 20%

    // Nível 2: Atributo estático (Contador)
    private static int totalColaboradores = 0;

    public FuncionarioBase(String nome) {
        this.nome = nome;
        totalColaboradores++;
    }
}
```

```

    }

    // Nível 2: Método Estático Utilitário (Cálculo de Imposto)
    public static double calcularImpostoSimples(double salario) {
        return salario * 0.11; // 11% fixo de retenção
    }

    public String getNome() { return nome; }
    public static int getTotalColaboradores() { return totalColaboradores; }

    // Nível 4: Garantir implementação com abstract
    public abstract double calcularSalarioLiquido();
}

// Nível 3: Hierarquia Abstrata (Classes Derivadas)
class GerenteSist extends FuncionarioBase {
    private double salarioBase;

    public GerenteSist(String nome, double salarioBase) {
        super(nome);
        // Validação: Não permite menor que o mínimo
        this.salarioBase = Math.max(salarioBase,
SALARIO_MINIMO);
    }

    @Override
    public double calcularSalarioLiquido() {
        double bruto = salarioBase + (salarioBase *
TAXA_BONUS_GERENTE);
        return bruto - calcularImpostoSimples(bruto);
    }
}

class VendedorSist extends FuncionarioBase {
    private double vendas;

```

```

        public VendedorSist(String nome, double vendas) {
            super(nome);
            this.vendas = vendas;
        }

        @Override
        public double calcularSalarioLiquido() {
            double comissao = vendas * 0.15;
            double bruto = Math.max(comissao,
SALARIO_MINIMO);
            return bruto - calcularImpostoSimples(bruto);
        }
    }

class EstagiarioSist extends FuncionarioBase {
    public EstagiarioSist(String nome) { super(nome); }

    @Override
    public double calcularSalarioLiquido() {
        // Estagiário recebe 80% do mínimo e não paga
imposto no exemplo
        return SALARIO_MINIMO * 0.8;
    }
}

// =====
// CAMADA DE EXECUÇÃO (Menu e Validações)
// =====

public class SistemaRHPro {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        ArrayList<FuncionarioBase> equipe = new
ArrayList<>();
        int opcao = 0;

        while (opcao != 3) {
            System.out.println("\n--- SISTEMA DE GESTÃO
PRO ---");
            System.out.println("1. Contratar
Colaborador");

```

```

        System.out.println("2. Ver Pannel de
Pagamentos");
        System.out.println("3. Sair");
        System.out.print("Escolha: ");

        // Validação de entrada numérica
        if (!leitor.hasNextInt()) {
            System.out.println("Erro: Digite apenas
números.");
            leitor.next(); continue;
        }
        opcao = leitor.nextInt();
        leitor.nextLine();

        switch (opcao) {
            case 1:
                System.out.println("\nCargos: 1-
Gerente | 2-Vendedor | 3-Estagiário");
                int cargo = leitor.nextInt();
                leitor.nextLine();

                System.out.print("Nome do
profissional: ");
                String nome = leitor.nextLine();

                // Nível 4 e 5: Aplicação de Regras
de Negócio

                if (cargo == 1) {
                    System.out.print("Salário Base
(Mín R$ " + FuncionarioBase.SALARIO_MINIMO + "): ");
                    double s = leitor.nextDouble();
                    equipe.add(new GerenteSist(nome,
s));
                } else if (cargo == 2) {
                    System.out.print("Volume de
Vendas: ");
                    double v = leitor.nextDouble();
                    equipe.add(new VendedorSist(nome,
v));
                } else if (cargo == 3) {
                    equipe.add(new
EstagiarioSist(nome));

```

```

        }
        System.out.println("Contratação
efetuada!");
        break;

        case 2:
            System.out.println("\n--- FOLHA
ANALÍTICA ---");
            for (FuncionarioBase f : equipe) {
                System.out.printf("Nome: %-12s |
Líquido: R$ %.2f\n",
                                f.getNome(),
                                f.calcularSalarioLiquido());
            }
            System.out.println("-----
-----");
            System.out.println("Total de Ativos:
" + FuncionarioBase.getTotalColaboradores());
            break;
        }
    }
    System.out.println("Encerrando...");
    leitor.close();
}
}
}

```

UNIP

CAUÃ ROCHA DE MORAES

PROGRAMAÇÃO ORIENTADA A OBJETOS

BRASÍLIA-DF 2025

1. Introdução

Conceito de Exceções

Uma exceção é um evento anormal que ocorre durante a execução de um programa e interrompe o fluxo normal de suas instruções. Diferente de um erro de sintaxe (que impede a compilação), a exceção acontece em tempo de execução (*runtime*), geralmente causada por dados de entrada inválidos, falhas de hardware ou indisponibilidade de recursos (como um arquivo não encontrado).

Importância do Tratamento

O tratamento de exceções é vital para garantir a **robustez** e a **confiabilidade** de um sistema. Sem ele, qualquer falha mínima causaria o fechamento abrupto do programa (*crash*), gerando uma experiência negativa para o usuário e possível perda de dados. Tratar exceções permite que o desenvolvedor:

- Capture o erro de forma elegante.
- Apresente uma mensagem amigável ao usuário.
- Tente recuperar o sistema ou finalizar processos (como fechar conexões de banco de dados) de forma segura.

2. Desenvolvimento

Explicação do Programa

O programa desenvolvido realiza operações de [ex: **Processamento de Dados Financeiros / Divisão de Valores**]. Por lidar com entradas diretas do usuário, o sistema está sujeito a falhas comuns, como a inserção de textos em campos numéricos ou tentativas de operações matematicamente impossíveis. A lógica de tratamento foi implementada para que o software valide essas situações em tempo real, mantendo-se ativo mesmo após a detecção de um erro.

Tipos de Exceções Utilizadas

No projeto, foram mapeadas as seguintes exceções específicas:

1. **ArithmeticException**: Acionada em erros de cálculos matemáticos, como a divisão de um número inteiro por zero.
2. **InputMismatchException**: Ocorre quando o usuário insere um tipo de dado diferente do esperado (ex: digitar uma letra quando o programa espera um número).
3. **Exception (Genérica)**: Utilizada como uma camada de segurança final para capturar qualquer outro erro imprevisto, evitando que o programa trave.

3. Código Comentado

O trecho abaixo demonstra o uso do bloco try-catch, que é a estrutura padrão para capturar e tratar possíveis falhas:

```
Java
try {
    // Bloco de código que pode gerar uma exceção
    int resultado = numerador / denominador;
    System.out.println("Resultado: " + resultado);

} catch (ArithmeticException e) {
    // Trata erro de divisão por zero
    // Impede que o programa pare de funcionar se o
    denominador for 0
    System.out.println("Erro: Não é possível realizar
    divisão por zero!");

} catch (Exception e) {
    // Captura qualquer outro erro inesperado
    System.out.println("Ocorreu um erro inesperado: " +
    e.getMessage());

} finally {
    // Bloco opcional que sempre será executado (útil
    para fechar arquivos/scanners)
    System.out.println("Operação finalizada.");
}
```

Código Fonte:

```
import java.util.ArrayList;
import java.util.Scanner;

// =====
// CAMADA DE MODELO (Abstração e Regras)
// =====

abstract class FuncionarioBase {
    private String nome;

    // Nível 1: Constantes (final e static)
    public static final double SALARIO_MINIMO = 1412.00;
    public static final double TAXA_BONUS_GERENTE = 0.20;
    // 20%

    // Nível 2: Atributo estático (Contador)
    private static int totalColaboradores = 0;

    public FuncionarioBase(String nome) {
        this.nome = nome;
        totalColaboradores++;
    }

    // Nível 2: Método Estático Utilitário (Cálculo de Imposto)
    public static double calcularImpostoSimples(double salario) {
        return salario * 0.11; // 11% fixo de retenção
    }

    public String getNome() { return nome; }
    public static int getTotalColaboradores() { return totalColaboradores; }

    // Nível 4: Garantir implementação com abstract
    public abstract double calcularSalarioLiquido();
}
```

```

// Nível 3: Hierarquia Abstrata (Classes Derivadas)
class GerenteSist extends FuncionarioBase {
    private double salarioBase;

    public GerenteSist(String nome, double salarioBase) {
        super(nome);
        // Validação: Não permite menor que o mínimo
        this.salarioBase = Math.max(salarioBase,
SALARIO_MINIMO);
    }

    @Override
    public double calcularSalarioLiquido() {
        double bruto = salarioBase + (salarioBase *
TAXA_BONUS_GERENTE);
        return bruto - calcularImpostoSimples(bruto);
    }
}

class VendedorSist extends FuncionarioBase {
    private double vendas;

    public VendedorSist(String nome, double vendas) {
        super(nome);
        this.vendas = vendas;
    }

    @Override
    public double calcularSalarioLiquido() {
        double comissao = vendas * 0.15;
        double bruto = Math.max(comissao,
SALARIO_MINIMO);
        return bruto - calcularImpostoSimples(bruto);
    }
}

class EstagiarioSist extends FuncionarioBase {
    public EstagiarioSist(String nome) { super(nome); }

    @Override

```

```

        public double calcularSalarioLiquido() {
            // Estagiário recebe 80% do mínimo e não paga
            imposto no exemplo
            return SALARIO_MINIMO * 0.8;
        }
    }

// =====
// CAMADA DE EXECUÇÃO (Menu e Validações)
// =====

public class SistemaRHPro {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        ArrayList<FuncionarioBase> equipe = new
ArrayList<>();
        int opcao = 0;

        while (opcao != 3) {
            System.out.println("\n--- SISTEMA DE GESTÃO
PRO ---");
            System.out.println("1. Contratar
Colaborador");
            System.out.println("2. Ver Painel de
Pagamentos");
            System.out.println("3. Sair");
            System.out.print("Escolha: ");

            // Validação de entrada numérica
            if (!leitor.hasNextInt()) {
                System.out.println("Erro: Digite apenas
números.");
                leitor.next(); continue;
            }
            opcao = leitor.nextInt();
            leitor.nextLine();

            switch (opcao) {
                case 1:
                    System.out.println("\nCargos: 1-
Gerente | 2-Vendedor | 3-Estagiário");
                    int cargo = leitor.nextInt();

```

```

        leitor.nextLine();

        System.out.print("Nome do
profissional: ");
        String nome = leitor.nextLine();

        // Nível 4 e 5: Aplicação de Regras
de Negócio

        if (cargo == 1) {
            System.out.print("Salário Base
(Mín R$ " + FuncionarioBase.SALARIO_MINIMO + "): ");
            double s = leitor.nextDouble();
            equipe.add(new GerenteSist(nome,
s));
        } else if (cargo == 2) {
            System.out.print("Volume de
Vendas: ");

            double v = leitor.nextDouble();
            equipe.add(new VendedorSist(nome,
v));
        } else if (cargo == 3) {
            equipe.add(new
EstagiarioSist(nome));
        }
        System.out.println("Contratação
efetuada!");
        break;

    case 2:
        System.out.println("\n--- FOLHA
ANALÍTICA ---");
        for (FuncionarioBase f : equipe) {
            System.out.printf("Nome: %-12s |
Líquido: R$ %.2f\n",
                                f.getNome(),
f.calcularSalarioLiquido());
        }
        System.out.println("-----
-----");

        System.out.println("Total de Ativos:
" + FuncionarioBase.getTotalColaboradores());
        break;

```

```
        }  
    }  
    System.out.println("Encerrando...");  
    leitor.close();  
}  
  
}
```

UNIP

CAUÃ ROCHA DE MORAES

PROGRAMAÇÃO ORIENTADA A OBJETOS

BRASÍLIA-DF 2025

1. Introdução

Conceito de Threads

Uma *Thread* (ou linha de execução) é a menor unidade de processamento que pode ser gerenciada por um sistema operacional. Enquanto um "processo" representa um programa em execução, uma *thread* é um fluxo de controle dentro desse processo. Um único programa pode ter múltiplas *threads* rodando simultaneamente, compartilhando o mesmo espaço de memória, o que torna a comunicação entre elas mais rápida do que entre processos distintos.

Execução Concorrente

A execução concorrente ocorre quando duas ou mais *threads* fazem progresso ao mesmo tempo. Em processadores com múltiplos núcleos, isso pode significar **paralelismo real** (cada thread em um núcleo). Em sistemas com apenas um núcleo, o sistema operacional alterna a execução entre as *threads* tão rapidamente que cria a ilusão de que tudo está acontecendo simultaneamente. Isso é essencial para manter interfaces gráficas responsivas enquanto o sistema realiza cálculos pesados em segundo plano.

2. Desenvolvimento

Explicação do Programa

O programa desenvolvido demonstra o uso de multitarefa ao realizar **[ex: o processamento de uma lista de dados / a simulação de um relógio e um cronômetro]** de forma independente. A lógica central permite que uma tarefa não precise esperar o término da outra para iniciar, otimizando o tempo total de execução. O sistema foi testado para garantir que o fluxo principal (*Main Thread*) permanecesse livre para interagir com o usuário enquanto as *threads* secundárias executavam suas funções específicas.

Tipo de Thread Utilizado

No projeto, a implementação seguiu o modelo de:

- **[Escolha a sua opção] Extensão da Classe Thread:** Criamos uma classe que herda de `java.lang.Thread`, sobrescrevendo o método `run()`. É a forma mais direta de criar uma thread.
- **[Escolha a sua opção] Implementação da Interface Runnable:** A classe implementa a interface `Runnable` e é passada como argumento para um objeto `Thread`. Este método é preferível na POO, pois permite que a classe herde de outras fontes.

Destaque de Implementação: Para iniciar a execução, utilizamos o método `.start()`. É importante notar que chamar `.run()` diretamente não cria uma nova thread, apenas

executa o código sequencialmente; o uso de `.start()` é o que solicita ao sistema operacional a criação do novo fluxo de execução.

Código Comentado (Exemplo para ilustração)

```
Java
// Implementação usando a interface Runnable
class TarefaSecundaria implements Runnable {
    @Override
    public void run() {
        // Código que será executado em paralelo
        System.out.println("Thread secundária em
execução...");
    }
}

public class Main {
    public static void main(String[] args) {
        TarefaSecundaria tarefa = new TarefaSecundaria();
        Thread thread1 = new Thread(tarefa);

        // Inicia a execução concorrente
        thread1.start();

        System.out.println("Thread principal continua
rodando.");
    }
}
```

Código Fonte:

```
import java.util.ArrayList;
import java.util.List;

/**
 * Nível 1: Implementação usando Runnable
 */
class ContadorTarefa implements Runnable {
    private final int inicio;
    private final int fim;
    private final int passo;
    private final int delay;

    public ContadorTarefa(int inicio, int fim, int passo,
int delay) {
        this.inicio = inicio;
        this.fim = fim;
        this.passo = passo;
        this.delay = delay;
    }

    @Override
    public void run() {
        // Nível 2: Nomeação capturada via
Thread.currentThread()
        String nomeThread =
Thread.currentThread().getName();

        for (int i = inicio; (passo > 0 ? i <= fim : i >=
fim); i += passo) {
            System.out.println "[" + nomeThread + "]
Valor: " + i);
            try {
                // Nível 3: Controle de tempo (delay
variável)

                Thread.sleep(delay);
            } catch (InterruptedException e) {
```

```

        System.out.println(nomeThread + " foi
interrompida.");
    }
}
    System.out.println(">>> " + nomeThread + "
finalizou!");
}
}

/**
 * Nível 5: Objeto para demonstrar Condição de Corrida
 */
class RecursoCompartilhado {
    public int contador = 0;

    // Método não sincronizado de propósito para observar
o erro
    public void incrementar() {
        contador++;
    }
}

public class MainMultithreading {
    public static void main(String[] args) throws
InterruptedException {
        System.out.println("=== INICIANDO EXECUÇÃO
CONCORRENTE ===\n");

        // --- PARTE 1: NÍVEIS 1 A 4 (EXECUÇÃO DE
TAREFAS) ---

        // Criando as instâncias de Runnable
        ContadorTarefa tarefaCrescente = new
ContadorTarefa(1, 10, 1, 400);
        ContadorTarefa tarefaDecrescente = new
ContadorTarefa(10, 1, -1, 400);
        ContadorTarefa tarefaRapida = new
ContadorTarefa(100, 110, 1, 200);

        // Criando as Threads
        Thread t1 = new Thread(tarefaCrescente);

```

```

Thread t2 = new Thread(tarefaDecrescente);
Thread t3 = new Thread(tarefaRapida);

// Nível 2: Nomeação
t1.setName("Thread A (Crescente)");
t2.setName("Thread B (Decrescente)");
t3.setName("Thread C (Rápida)");

// Nível 4: Start nas múltiplas threads
t1.start();
t2.start();
t3.start();

// Aguarda as threads de contagem terminarem
antes de iniciar o desafio extra
t1.join();
t2.join();
t3.join();

// --- PARTE 2: NÍVEL 5 (DESAFIO DE
SINCRONIZAÇÃO) ---

System.out.println("\n=== DESAFIO: CONDIÇÃO DE
CORRIDA ===");
RecursoCompartilhado recurso = new
RecursoCompartilhado();
int totalThreads = 20;
int incrementosPorThread = 1000;

List<Thread> threadsCorrida = new ArrayList<>();

Runnable tarefaSoma = () -> {
    for (int i = 0; i < incrementosPorThread;
i++) {
        recurso.incrementar();
    }
};

// Dispara 20 threads somando ao mesmo tempo
for (int i = 0; i < totalThreads; i++) {
    Thread t = new Thread(tarefaSoma);

```

```
        threadsCorrida.add(t);
        t.start();
    }

    // Aguarda todas as threads de soma terminarem
    for (Thread t : threadsCorrida) t.join();

    int esperado = totalThreads *
incrementosPorThread;
    System.out.println("Resultado Esperado: " +
esperado);
    System.out.println("Resultado Real:      " +
recurso.contador);

    if (recurso.contador < esperado) {
        System.out.println("STATUS: Inconsistência
detectada! (As threads se atropelaram)");
    } else {
        System.out.println("STATUS: Resultado
consistente (Sorte do escalonador)");
    }
}
}
```